

Guía Práctica de Git y GitHub para Ciencia de Datos

Comandos esenciales y flujos de trabajo

Autor: Ernesto Alfonso Ruiz Laya Castillo

Entorno: Canaima GNU/Linux, VS Code

Versión: 1.0 - Enero 2026

Resumen

Esta guía cubre los comandos esenciales de Git y GitHub para proyectos de ciencia de datos. Incluye configuración inicial, flujo de trabajo básico, manejo de archivos, actualizaciones y resolución de conflictos comunes.

Índice

1. Introducción	3
1.1. ¿Qué es Git?	3
1.2. ¿Qué es GitHub?	3
1.3. Flujo de trabajo típico en Ciencia de Datos	3
2. Configuración Inicial	4
2.1. Instalación y configuración básica	4
2.2. Configuración para proyectos Python	4
3. Flujo de Trabajo Básico	5
3.1. Crear un nuevo repositorio	5
3.2. Estructura recomendada para proyectos de datos	5
4. Comandos Esenciales	5
4.1. Añadir archivos nuevos	5
4.2. Confirmar cambios (commit)	7
4.3. Subir cambios a GitHub	7
5. Manejo de Archivos Existentes	8
5.1. Modificar archivos ya subidos	8
5.2. Eliminar archivos	9
5.3. Renombrar o mover archivos	9
6. Actualizar desde GitHub	10
6.1. Obtener cambios del repositorio remoto	10
7. Resolución de Conflictos	11
7.1. Conflictos comunes y soluciones	11

8. Configuración Específica para Ciencia de Datos	12
8.1. Archivo .gitignore optimizado	12
8.2. Gestión de datasets grandes	13
9. Flujos de Trabajo Avanzados	14
9.1. Convenciones de commits	14
9.2. Tags y versiones	14
10. Comandos Útiles y Aliases	15
10.1. Aliases para mayor productividad	15
10.2. Comandos de diagnóstico	16
11. Apéndices	17
11.1. Errores comunes y soluciones	17
11.2. Recursos adicionales	17
12. Conclusión	18

1 Introducción

1.1 ¿Qué es Git?

Git es un sistema de control de versiones distribuido que permite:

- Seguir cambios en archivos de código
- Trabajar en equipo de manera coordinada
- Mantener historial completo de modificaciones
- Crear ramas para experimentar sin afectar el código principal

1.2 ¿Qué es GitHub?

GitHub es una plataforma que aloja repositorios Git en la nube, proporcionando:

- Almacenamiento remoto de código
- Colaboración mediante pull requests
- Issue tracking
- GitHub Actions para CI/CD

1.3 Flujo de trabajo típico en Ciencia de Datos

1. Inicializar repositorio local
2. Crear estructura del proyecto
3. Añadir archivos (`add`)
4. Confirmar cambios (`commit`)
5. Subir a GitHub (`push`)
6. Actualizar desde GitHub (`pull`)

2 Configuración Inicial

2.1 Instalación y configuración básica

```
1 # Verificar instalaci n
2 git --version
3
4 # Configurar usuario (solo primera vez)
5 git config --global user.name "Tu_Nombre"
6 git config --global user.email "tu.email@ejemplo.com"
7
8 # Configurar editor preferido
9 git config --global core.editor "code_--wait"
10
11 # Ver configuraci n actual
12 git config --list
```

3 Flujo de Trabajo Básico

3.1 Crear un nuevo repositorio

```
1 # Opción 1: Crear nuevo repositorio local
2 mkdir mi_proyecto_datos
3 cd mi_proyecto_datos
4 git init
5
6 # Opción 2: Clonar repositorio existente
7 git clone https://github.com/usuario/repositorio.git
8 cd repositorio
```

Listing 3: Inicializar repositorio desde cero

3.2 Estructura recomendada para proyectos de datos

```
1 mi_proyecto_datos/
2     .gitignore           # Archivos a ignorar
3     README.md            # Documentación
4     requirements.txt      # Dependencias
5     data/                # Datasets
6         raw/              # Datos originales
7         processed/        # Datos procesados
8     notebooks/           # Jupyter notebooks
9     scripts/             # Scripts Python
10         modulo_1/
11         modulo_2/
12     src/                 # Código fuente
13     tests/               # Pruebas
```

Listing 4: Estructura de carpetas típica

4 Comandos Esenciales

4.1 Añadir archivos nuevos

```
1 # Ver estado actual
2 git status
3
4 # Añadir archivo específico
5 git add scripts/modulo_1/01_variables.py
6
7 # Añadir todos los archivos nuevos y modificados
8 git add .
9
10 # Añadir todos los archivos Python
11 git add *.py
12
```

```
13 # Aadir carpeta completa
14 git add scripts/modulo_2/
15
16 # Ver qu se ha a adido
17 git status
```

Listing 5: Añadir archivos al staging area

4.2 Confirmar cambios (commit)

```
1 # Commit básico
2 git commit -m "Agrega análisis inicial de datos"
3
4 # Commit con mensaje detallado
5 git commit -m "feat: agrega modelo de regresión lineal
6
7     - Implementa LinearRegression de scikit-learn
8     - Agrega validación cruzada
9     - Incluye métricas de evaluación (MAE, RMSE, R )
10    - Documenta parámetros del modelo"
11
12 # Corregir último commit (si olvidaste algo)
13 git add archivo_olvidado.py
14 git commit --amend -m "Mensaje corregido"
15
16 # Ver historial de commits
17 git log --oneline -5
```

Listing 6: Crear commits descriptivos

4.3 Subir cambios a GitHub

```
1 # Conectar repositorio local con GitHub (solo primera vez)
2 git remote add origin https://github.com/usuario/repositorio.git
3
4 # Subir cambios
5 git push origin main
6
7 # Subir cambios y establecer upstream
8 git push -u origin main
9
10 # Subir etiquetas (tags)
11 git push origin --tags
```

Listing 7: Sincronizar con repositorio remoto

5 Manejo de Archivos Existentes

5.1 Modificar archivos ya subidos

```
1 # 1. Modificar el archivo (ej: scripts/modulo_2/01_intro_numpy.py)
2
3 # 2. Ver cambios realizados
4 git diff scripts/modulo_2/01_intro_numpy.py
5
6 # 3. A adir cambios al staging
7 git add scripts/modulo_2/01_intro_numpy.py
8
9 # 4. Confirmar cambios
10 git commit -m "fix: corrige condición de filtro en análisis de
11     presi n"
12
13 # 5. Subir cambios
14 git push origin main
```

Listing 8: Actualizar archivos existentes

5.2 Eliminar archivos

```
1 # Eliminar archivo local y del repositorio
2 git rm archivo_obsoleto.py
3 git commit -m "remove: elimina archivo obsoleto"
4 git push origin main
5
6 # Eliminar solo del repositorio (mantener local)
7 git rm --cached archivo_sensible.csv
8 git commit -m "remove: elimina dataset sensible del tracking"
9 git push origin main
```

Listing 9: Eliminar archivos del repositorio

5.3 Renombrar o mover archivos

```
1 # Renombrar archivo
2 git mv viejo_nombre.py nuevo_nombre.py
3 git commit -m "refactor: renombra script para mayor claridad"
4 git push origin main
5
6 # Mover archivo a otra carpeta
7 git mv script.py scripts/modulo_1/script.py
8 git commit -m "chore: reorganiza estructura de carpetas"
9 git push origin main
```

Listing 10: Renombrar archivos manteniendo historial

6 Actualizar desde GitHub

6.1 Obtener cambios del repositorio remoto

```
1 # Descargar cambios sin aplicar
2 git fetch origin
3
4 # Ver diferencias con remoto
5 git diff main origin/main
6
7 # Traer y aplicar cambios (pull = fetch + merge)
8 git pull origin main
9
10 # Traer cambios con rebase (historial m s limpio)
11 git pull --rebase origin main
```

Listing 11: Sincronizar con cambios remotos

7 Resolución de Conflictos

7.1 Conflictos comunes y soluciones

```
1 # Cuando git pull muestra conflictos:
2 git pull origin main
3 # Si hay conflictos, Git marca los archivos
4
5 # Ver archivos en conflicto
6 git status
7
8 # Resolver conflictos manualmente en VS Code
9 # Los conflictos aparecen como:
10 # <<<<<< HEAD
11 # c digo local
12 # =====
13 # c digo remoto
14 # >>>>>> branch-name
15
16 # Después de resolver:
17 git add archivo_resuelto.py
18 git commit -m "merge: resuelve conflictos en archivo.py"
19 git push origin main
```

Listing 12: Manejo de conflictos de merge

8 Configuración Específica para Ciencia de Datos

8.1 Archivo .gitignore optimizado

```
1 # Entornos virtuales
2 env*/
3 venv/
4 .env
5 *.env
6
7 # Datos (no versionar datasets grandes)
8 data/
9 *.csv
10 *.xlsx
11 *.json
12 *.pkl
13 *.h5
14 *.feather
15 *.parquet
16
17 # Notebooks
18 .ipynb_checkpoints/
19 *.ipynb_checkpoints/
20
21 # Python
22 __pycache__/*
23 *.py[cod]
24 *$py.class
25 *.so
26 .Python
27
28 # VS Code
29 .vscode/
30 !.vscode/settings.json
31 !.vscode/launch.json
32 !.vscode/tasks.json
33
34 # Sistema
35 .DS_Store
36 Thumbs.db
37 desktop.ini
38
39 # Logs y archivos temporales
40 *.log
41 *.tmp
42 *.temp
```

Listing 13: .gitignore para proyectos Python/Data Science

8.2 Gestión de datasets grandes

```
1 # Usar Git LFS para datasets grandes
2 # Instalar Git LFS primero
3 git lfs install
4
5 # Especificar tipos de archivos para LFS
6 git lfs track "*.csv"
7 git lfs track "*.pkl"
8 git lfs track "data/**"
9
10 # Asegurarse de trackear el archivo .gitattributes
11 git add .gitattributes
12 git commit -m "chore: configura Git LFS para datasets"
```

Listing 14: Manejo de archivos grandes

9 Flujos de Trabajo Avanzados

9.1 Convenciones de commits

```
1 # Estructura: tipo( mbito ): descripci n
2
3 # Tipos comunes:
4 # feat: nueva funcionalidad
5 # fix: correcci n de bug
6 # docs: documentaci n
7 # style: formato (no afecta c digo)
8 # refactor: refactorizaci n
9 # test: pruebas
10 # chore: tareas de mantenimiento
11
12 # Ejemplos:
13 git commit -m "feat(analisis): agrega clustering con K-means"
14 git commit -m "fix(modelo): corrige c lculo de precisi n"
15 git commit -m "docs(readme): actualiza instrucciones de instalaci n"
16 git commit -m "test(regresion): agrega pruebas unitarias"
```

Listing 15: Convenciones para mensajes de commit

9.2 Tags y versiones

```
1 # Crear tag para versi n
2 git tag -a v1.0.0 -m "Versi n 1.0.0: An lisis completo"
3 git push origin v1.0.0
4
5 # Listar tags
6 git tag
7
8 # Crear tag para commit espec fico
9 git tag -a v0.1.0 <commit-hash> -m "Versi n inicial"
```

Listing 16: Etiquetado de versiones

Listing 17: Aliases útiles en `/.gitconfig`

10.2 Comandos de diagnóstico

```
1 # Ver espacio usado por Git
2 git count-objects -vH
3
4 # Limpiar archivos no trackeados
5 git clean -n # Ver qu  se eliminar a
6 git clean -f # Eliminar archivos no trackeados
7 git clean -fd # Eliminar archivos y directorios
8
9 # Ver historial gr fico
10 git log --oneline --graph --all --decorate
11
12 # Buscar en el historial
13 git log --grep="análisis" # Commits con "análisis"
14 git log -S "import pandas" # Commits que a aden/remueven c digo
```

Listing 18: Diagnóstico y limpieza

11 Apéndices

11.1 Errores comunes y soluciones

Error	Solución
<code>fatal: not a git repository</code>	Ejecutar <code>git init</code> o navegar a carpeta correcta
<code>support for password authentication was removed</code>	Usar tokens de acceso personal en GitHub
<code>updates were rejected</code>	Hacer <code>git pull</code> primero, resolver conflictos
<code>Your branch is ahead of 'origin/main'</code>	Ejecutar <code>git push origin main</code>
<code>Please commit your changes or stash them</code>	Guardar cambios con <code>git stash</code> o hacer <code>commit</code>
<code>Large files detected</code>	Usar Git LFS o añadir al <code>.gitignore</code>

11.2 Recursos adicionales

- Oficial: <https://git-scm.com/doc>
- GitHub Guides: <https://guides.github.com>
- Git LFS: <https://git-lfs.github.com>
- Conventional Commits: <https://www.conventionalcommits.org>
- Git Flow: <https://nvie.com/posts/a-successful-git-branching-model>

12 Conclusión

Esta guía cubre los comandos esenciales para trabajar con Git y GitHub en proyectos de ciencia de datos. Recuerda:

- **Commit temprano y frecuente:** Pequeños commits descriptivos
- **Mensajes claros:** Usar convenciones de commits
- **No versionar datos:** Usar `.gitignore` para datasets
- **Sincronizar regularmente:** `pull` antes de `push`
- **Documentar:** `README.md` actualizado

Para dudas específicas, consulta la documentación oficial o pregunta en nuestro canal de comunicación.

Documento actualizado: Enero 2026
Específico para entorno Canaima GNU/Linux y VS Code